Ashwitha.R

2303A51340

1)

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();

        int k = sc.nextInt();
        int max = 0;

        for (int i = 0; i < n; i++) {
            int sum = 0;

            for (int j = i; j < n; j++) {
                sum += arr[j];

                if (sum == k)
                    max = Math.max(max, j - i + 1);
            }
        }

        System.out.println(max);
    }
}
```

2)

```
import java.util.*;
```

```
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String s = sc.nextLine();
        String t = sc.nextLine();

        String ans = "";
        int min = Integer.MAX_VALUE;

        for (int i = 0; i < s.length(); i++) {
            for (int j = i; j < s.length(); j++) {
                String sub = s.substring(i, j + 1);
                int[] freq = new int[65536];

                for (int x = 0; x < sub.length(); x++)
                    freq[sub.charAt(x)]++;

                boolean found = true;

                for (int x = 0; x < t.length(); x++) {
                    char c = t.charAt(x);

                    if (freq[c] == 0) {
                        found = false;
                        break;
                    }

                    freq[c]--;
                }

                if (found && sub.length() < min) {
                    min = sub.length();
```

```
                ans = sub;
            }
        }
    }

    System.out.println(ans);
  }
}
```

3)

```
import java.util.*;

class Main {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    int[][] a = new int[n][2];

    for (int i = 0; i < n; i++) {
      a[i][0] = sc.nextInt();
      a[i][1] = sc.nextInt();
    }

    boolean changed = true;

    while (changed) {
      changed = false;

      for (int i = 0; i < n; i++) {
        if (a[i][0] == Integer.MAX_VALUE)
          continue;

        for (int j = i + 1; j < n; j++) {
          if (a[j][0] == Integer.MAX_VALUE)
```

```
                continue;

            if (a[i][0] <= a[j][1]
                && a[j][0] <= a[i][1]) {
                a[i][0] = Math.min(a[i][0], a[j][0]);
                a[i][1] = Math.max(a[i][1], a[j][1]);

                a[j][0] = Integer.MAX_VALUE;
                changed = true;
                break;
            }
          }

          if (changed)
            break;
        }
      }

      for (int i = 0; i < n; i++) {
        if (a[i][0] != Integer.MAX_VALUE)
          System.out.println(a[i][0] + " " + a[i][1]);
      }
    }
}
```

4)

```
import java.util.*;

class Main {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    ArrayList<Integer> list = new ArrayList<>();
```

```
        for (int i = 0; i < n; i++)
            list.add(sc.nextInt());

        int k = sc.nextInt();
        int q = sc.nextInt();

        for (int i = 0; i < q; i++) {
            int val = sc.nextInt();
            list.add(val);

            Collections.sort(list, Collections.reverseOrder());

            if (list.size() >= k)
                System.out.println(list.get(k - 1));
            else
                System.out.println(-1);
        }
    }
}
```

5)

```
import java.util.*;

class Main {
    static class Node {
        int data;
        Node next;

        Node(int data) {
            this.data = data;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
int n = sc.nextInt();
Node head = null, tail = null;
Node[] nodes = new Node[n];

for (int i = 0; i < n; i++) {
   nodes[i] = new Node(sc.nextInt());

   if (head == null)
      head = nodes[i];
   else
      tail.next = nodes[i];

   tail = nodes[i];
}

int pos = sc.nextInt();

if (n > 0 && pos >= 0 && pos < n)
   tail.next = nodes[pos];

Node curr = head;
HashSet<Node> set = new HashSet<>();
Node prev = null;

while (curr != null && !set.contains(curr)) {
   set.add(curr);
   prev = curr;
   curr = curr.next;
}

if (curr != null)
   prev.next = null;
```

```
        curr = head;

        while (curr != null) {
            System.out.print(curr.data + " ");
            curr = curr.next;
        }
    }
}
```

6)

```
import java.util.*;

class Main {
    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
        }
    }

    static boolean findPath(Node root, int value,
                            ArrayList<Node> path) {
        if (root == null)
            return false;

        path.add(root);

        if (root.data == value)
            return true;

        if (findPath(root.left, value, path)
                || findPath(root.right, value, path))
```

```
        return true;

    path.remove(path.size() - 1);
    return false;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();

    if (n == 0) {
        System.out.println(-1);
        return;
    }

    int[] a = new int[n];

    for (int i = 0; i < n; i++)
        a[i] = sc.nextInt();

    Node root = null;
    Queue<Node> q = new LinkedList<>();

    if (a[0] != -1) {
        root = new Node(a[0]);
        q.offer(root);
    }

    int i = 1;

    while (!q.isEmpty() && i < n) {
        Node curr = q.poll();
```

```
    if (i < n && a[i] != -1) {
        curr.left = new Node(a[i]);
        q.offer(curr.left);
    }
    i++;

    if (i < n && a[i] != -1) {
        curr.right = new Node(a[i]);
        q.offer(curr.right);
    }
    i++;
}

int p = sc.nextInt();
int r = sc.nextInt();

ArrayList<Node> path1 = new ArrayList<>();
ArrayList<Node> path2 = new ArrayList<>();

if (!findPath(root, p, path1)
        || !findPath(root, r, path2)) {
    System.out.println(-1);
    return;
}

int ans = -1;

for (int j = 0; j < Math.min(path1.size(), path2.size()); j++) {
    if (path1.get(j) == path2.get(j))
        ans = path1.get(j).data;
    else
        break;
```

```
        }

        System.out.println(ans);
    }
}
```

7)

```
import java.util.*;

class Main {
    static int min = Integer.MAX_VALUE;

    static void findPath(int[][] a, boolean[][] visited,
                         int r, int c, int len) {
        int n = a.length;

        if (len >= min)
            return;

        if (r == n - 1 && c == n - 1) {
            min = len;
            return;
        }

        visited[r][c] = true;

        for (int dr = -1; dr <= 1; dr++) {
            for (int dc = -1; dc <= 1; dc++) {
                int nr = r + dr;
                int nc = c + dc;

                if (nr >= 0 && nr < n && nc >= 0 && nc < n
                        && a[nr][nc] == 0
                        && !visited[nr][nc]) {
                    findPath(a, visited, nr, nc, len + 1);
```

```
            }
          }
        }

        visited[r][c] = false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[][] a = new int[n][n];

        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();

        if (n == 0 || a[0][0] != 0 || a[n - 1][n - 1] != 0) {
            System.out.println(-1);
            return;
        }

        boolean[][] visited = new boolean[n][n];
        findPath(a, visited, 0, 0, 1);

        System.out.println(min == Integer.MAX_VALUE ? -1 : min);
    }
}
```

8)

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
int n = sc.nextInt();
int m = sc.nextInt();

int[][] pre = new int[m][2];

for (int i = 0; i < m; i++) {
   pre[i][0] = sc.nextInt();
   pre[i][1] = sc.nextInt();
}

boolean[] done = new boolean[n];
int[] order = new int[n];
int count = 0;

while (count < n) {
   boolean progress = false;

   for (int course = 0; course < n; course++) {
      if (done[course])
         continue;

      boolean canTake = true;

      for (int j = 0; j < m; j++) {
         if (pre[j][0] == course
               && !done[pre[j][1]]) {
            canTake = false;
            break;
         }
      }

      if (canTake) {
```

```
                done[course] = true;
                order[count++] = course;
                progress = true;
            }
        }

        if (!progress) {
            System.out.println("[]");
            return;
        }
    }

    for (int i = 0; i < n; i++)
        System.out.print(order[i] + " ");
  }
}
```

```
9) import java.util.*;

class Main {
   static int solve(int[] coins, int amount) {
      if (amount == 0)
         return 0;

      if (amount < 0)
         return Integer.MAX_VALUE;

      int min = Integer.MAX_VALUE;

      for (int coin : coins) {
         int result = solve(coins, amount - coin);

         if (result != Integer.MAX_VALUE)
            min = Math.min(min, result + 1);
```

```
        }

        return min;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] coins = new int[n];

        for (int i = 0; i < n; i++)
            coins[i] = sc.nextInt();

        int amount = sc.nextInt();

        int ans = solve(coins, amount);

        System.out.println(ans == Integer.MAX_VALUE ? -1 : ans);
    }
}
```

10)

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] a = new int[n];

        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();
```

```
        int q = sc.nextInt();

        for (int i = 0; i < q; i++) {
            int type = sc.nextInt();

            if (type == 1) {
                int index = sc.nextInt();
                int value = sc.nextInt();

                a[index] = value;
            } else if (type == 2) {
                int left = sc.nextInt();
                int right = sc.nextInt();

                int max = Integer.MIN_VALUE;

                for (int j = left; j <= right; j++)
                    max = Math.max(max, a[j]);

                System.out.println(max);
            }
        }
    }
}
```